PROJECT OVERVIEW

RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop (HITL)

High-Level Design Document

Name : Eesha Haldavnekar

Intern ID : IN226120202

Made with GAMMA

System Overview

Problem Statement

Traditional customer support systems suffer from slow response times, inconsistent answers, and high operational costs due to manual handling of repetitive queries across multiple support channels.

Solution

An AI-powered Retrieval-Augmented Generation (RAG) system that leverages a PDF knowledge base to provide instant, contextually accurate responses while maintaining human oversight for complex or sensitive queries.

Scope

- Answer user queries using document context from uploaded PDFs
- Automatically route simple queries to AI and complex queries to human agents
- Maintain conversation history and escalation trails
- Support iterative refinement of responses through human feedback

Made with GAMMA

Architecture Overview

System Architecture Components

The system follows a modular pipeline architecture with the following layers:

Input Layer

- User Interface (CLI): Command-line interface for user queries and document uploads

Data Processing Layer

- PDF Document Loader: Extracts text from uploaded PDF documents
- Text Chunking Module: Splits documents into overlapping chunks (400 tokens, 50-token overlap)
- Embedding Model: sentence-transformers/all-MiniLM-L6-v2 converts text chunks to 384-dimensional vectors

Storage & Retrieval Layer

- Vector Database (ChromaDB): Stores embeddings with metadata for fast similarity search
- Retriever: Performs semantic search to fetch top-k relevant chunks based on query embeddings

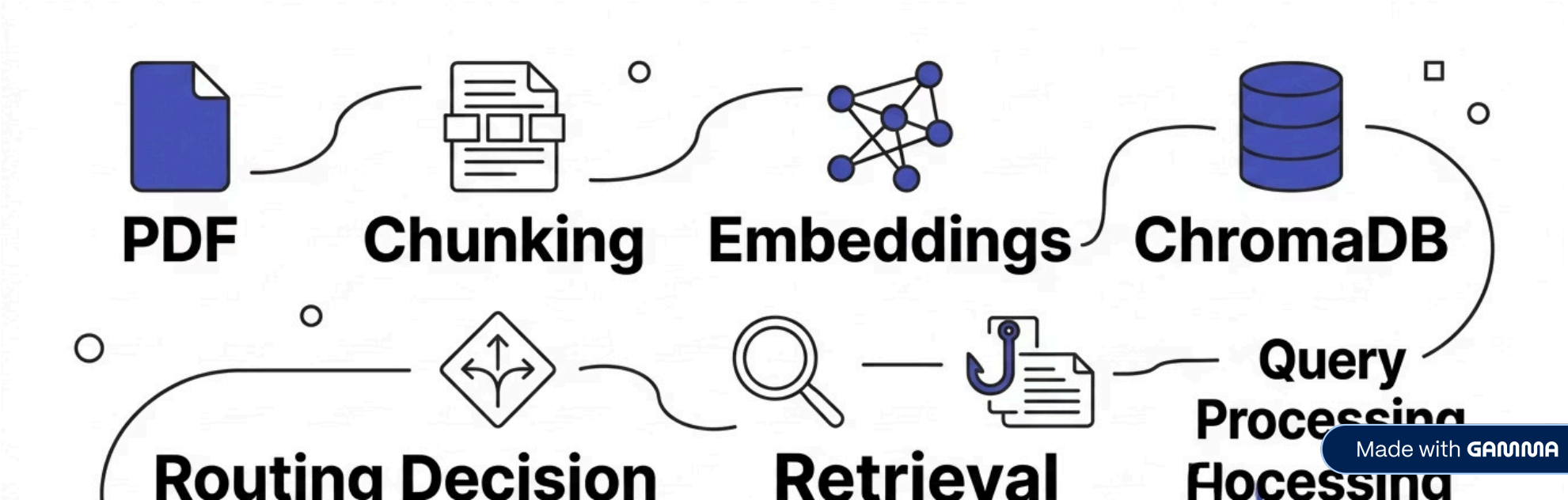
Intelligence Layer

- LLM (LLaMA via Ollama): Generates contextually accurate responses from retrieved documents
- Routing Logic: Determines if query requires AI response or human escalation
- LangGraph Workflow Engine: Orchestrates the entire workflow as a directed acyclic graph

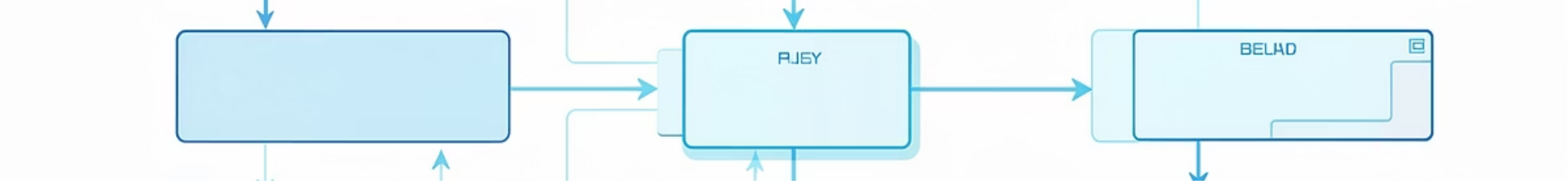
Human Oversight Layer

- Human-in-the-Loop (HITL) Module: Routes complex queries to human agents, collects feedback, and refines responses

Data Flow Path



Made with GAMMA



Component Descriptions

1. Document Loader

Responsible for ingesting PDF files and extracting raw text. Uses PyPDF2 or pdfplumber to parse documents while preserving structure and metadata. Handles multi-page documents and maintains document source references for traceability.

2. Text Chunking Strategy

Implements intelligent document segmentation with:

- Chunk Size: 400 tokens per chunk
- Overlap: 50 tokens between consecutive chunks
- Strategy: Semantic-aware chunking that respects sentence boundaries
- Benefit: Maintains context while enabling efficient retrieval and reducing token consumption

3. Embedding Model (sentence-transformers/all-MiniLM-L6-v2)

Lightweight transformer model that converts text into 384-dimensional dense vectors. Optimized for semantic similarity tasks with fast inference. Chosen for its balance between performance and computational efficiency, suitable for CPU-based deployments.

4. Vector Store (ChromaDB)

In-memory vector database with persistent storage capabilities. Stores embeddings alongside metadata (document source, chunk position). Provides fast approximate nearest neighbor search using HNSW algorithm. Lightweight alternative to enterprise solutions, ideal for prototyping and small-to-medium deployments.

5. Retriever

Performs semantic similarity search by converting user queries to embeddings and finding top-k most similar chunks from ChromaDB. Configurable similarity threshold and result count. Returns ranked results with relevance scores for downstream processing.

6. LLM (LLaMA via Ollama)

Open-source large language model running locally via Ollama. Eliminates API dependencies and ensures data privacy. Supports context windows up to 4K tokens. Generates responses based on retrieved context and user queries without hallucinating beyond provided documents.

7. LangGraph Workflow Engine

Orchestrates the entire pipeline as a directed graph with nodes representing processing steps and edges representing data flow. Enables conditional routing, error handling, and state management. Supports visualization and debugging of workflow execution.

8. Routing Layer

Decision logic that evaluates query complexity and confidence scores:

- High confidence + simple query → Direct LLM response
- Low confidence or complex query → Route to human agent
- Maintains escalation history and reasoning for audit trails

9. Human-in-the-Loop (HITL) Module

Made with GAMMA

Data Flow & Processing Pipeline

End-to-End Data Flow

Step 1: Document Ingestion

User uploads PDF documents through the CLI interface. The Document Loader extracts raw text while preserving document metadata (filename, upload timestamp, source).

Step 2: Text Chunking

Raw text is split into overlapping chunks of 400 tokens with 50-token overlap. This strategy ensures context preservation across chunk boundaries while maintaining manageable token counts for embedding and retrieval.

Step 3: Embedding Generation

Each chunk is converted to a 384-dimensional vector using the sentence-transformers/all-MiniLM-L6-v2 model. Embeddings capture semantic meaning, enabling similarity-based retrieval.

Step 4: Vector Storage

Embeddings and their associated metadata (chunk text, document source, position) are stored in ChromaDB. The database indexes embeddings using HNSW for efficient similarity search.

Step 5: Query Processing

User submits a query through the CLI. The query is converted to an embedding using the same embedding model, ensuring semantic alignment with stored chunks.

Step 6: Semantic Retrieval

The Retriever performs similarity search in ChromaDB, returning top-k chunks (typically 3-5) with highest relevance scores. Retrieved chunks form the context for the LLM.

Step 7: Routing Decision

The Routing Layer evaluates:

- Confidence score from retriever (relevance of retrieved chunks)
- Query complexity heuristics (length, entity count, question type)
- Historical escalation patterns

Step 8: Response Generation or Escalation

- If routed to AI: LLM generates response using retrieved context and user query
- If routed to Human: Query is queued for human agent with context and confidence metrics

Step 9: Response Delivery & Feedback

Response is delivered to user. Human feedback (if applicable) is collected and stored for continuous improvement of routing logic and model fine-tuning.

Made with GAMMA

Technology Choices & Rationale

Why ChromaDB?

- Lightweight and easy to deploy without external infrastructure
- Built-in support for metadata filtering and hybrid search
- Persistent storage with in-memory performance
- Active community and regular updates
- Ideal for prototyping and production use cases at scale
- No vendor lock-in compared to cloud-based vector databases

Why LangGraph?

- Purpose-built for orchestrating LLM workflows with complex logic
- Native support for state management and conditional routing
- Enables visualization of workflow execution for debugging
- Integrates seamlessly with LangChain ecosystem
- Supports human-in-the-loop patterns natively
- Reduces boilerplate code compared to custom orchestration

Why Ollama (LLaMA)?

- Runs entirely on-premises, ensuring data privacy and compliance
- No API rate limits or external dependencies
- Cost-effective compared to cloud LLM APIs
- Supports multiple model sizes (7B, 13B, 70B) for flexibility
- Fast inference with GPU acceleration support
- Community-driven with extensive model library

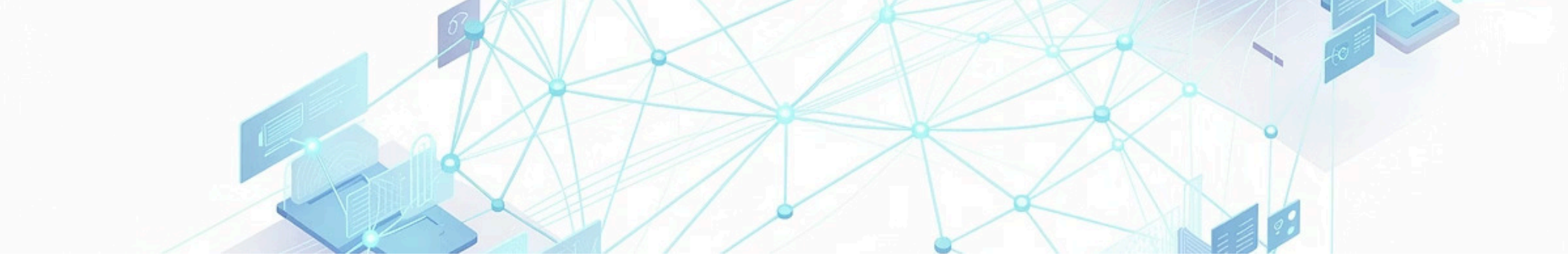
Why HuggingFace Embeddings (sentence-transformers/all-MiniLM-L6-v2)?

- Lightweight model (22M parameters) with fast inference
- 384-dimensional embeddings provide good semantic representation
- Optimized for semantic similarity tasks
- No external API calls required (runs locally)
- Excellent performance-to-size ratio
- Pre-trained on diverse datasets ensuring broad domain coverage
- Open-source with no licensing restrictions

Integration Benefits

These technologies form a cohesive, privacy-first stack that eliminates external dependencies while maintaining production-grade performance. The combination enables rapid iteration, easy debugging, and seamless scaling.

Made with GAMMA



Scalability Considerations

Handling Large Documents

- Implement batch processing for document ingestion to avoid memory spikes
- Use streaming chunking for documents exceeding 100MB
- Implement document versioning to track updates and maintain consistency
- Consider distributed chunking for very large document collections (>10GB)
- Archive old documents to separate storage while maintaining searchability

Managing Query Load

- Implement query caching to reduce redundant embedding computations
- Use connection pooling for ChromaDB to handle concurrent requests
- Deploy multiple LLM instances behind a load balancer for horizontal scaling
- Implement request queuing with priority levels for escalated queries
- Monitor query latency and implement adaptive timeout mechanisms

Latency Optimization

- Pre-compute embeddings during off-peak hours for frequently accessed documents
- Implement approximate nearest neighbor search with configurable precision
- Use GPU acceleration for embedding generation and LLM inference
- Cache frequently retrieved chunks in memory for sub-millisecond access
- Implement response streaming to provide partial results while processing continues
- Monitor and optimize chunk size based on retrieval performance metrics

Infrastructure Scaling Strategy

- Containerize components using Docker for easy deployment and scaling
- Use Kubernetes for orchestration and auto-scaling based on load metrics
- Implement database replication for ChromaDB to ensure high availability
- Separate read and write operations for vector database optimization
- Monitor system metrics (CPU, memory, latency) and trigger auto-scaling at thresholds

Future Enhancements

- Migrate to distributed vector databases (Milvus, Weaviate) for multi-node deployments
- Implement model quantization to reduce LLM memory footprint
- Add support for multi-modal embeddings (text + images)
- Implement federated learning for privacy-preserving model improvements
- Consider hybrid search combining keyword and semantic search for better recall

Made with GAMMA